

Users:

- **Society / Apartment Committee Members:**
Define and manage maintenance rates and space-type charge rules.
- **Treasurer / Accountant:**
Record payments, track dues, generate receipts, and maintain financial transparency.
- **Flat / Shop Owners (Residents):**
View their charges, payment status, and receipts; ensure accuracy and accountability.
- **Auditors:**
Review and verify maintenance and payment records for compliance and reporting.

Use Case:

Maintain transparent, organized, and up-to-date records for maintenance billing, payments, and receipts across multiple apartments or shops.
Manual entry through spreadsheets or paper logs often leads to inconsistent charge calculations, missed updates, and delayed reconciliation.

This application provides a centralized system that:

- Defines customizable maintenance charges by space type (e.g., 1BHK, 2BHK, Shop).
- Applies those charges automatically to each registered member’s flat.
- Records and tracks payments with penalties, balances, and receipts.
- Enables quick viewing, editing, and printing of invoices and records with real-time data updates.

User Features:

Define Charges: *Allows customizable general charges for different user-defined space types.*

- **Add various user-defined space types;** in accordance to the apartment size.(eg: 1BHK, 2BHK, SHOP)
- **Includes pre-defined base charges;** which are applicable to the specified space types(building maintenance, repair funds and contingency funds).
- **Specify parking rates;** users can set parking rates for any number of vehicles or a standard parking rate for all.
- **Specify lease/rent charges;** applicable to each space type.
- **Add/remove additional charges;** for varying purposes(eg: utilities, security charges, cleaning charges, amenities)
- The payable amount for all charges can be changed.
- **Add/remove rows** to make new definitions or **delete existing rows.**
- **Bulk deletion;** delete multiple rows with a single click.
- **Built in data validation;** ensures required fields are filled.
- **Auto save/update;** all changes save immediately, no saves required.
- **Auto refresh;** no manual refresh needed, but still present.

Define Resident’s Maintenance charges: *Record resident’s (maintenance payer) details to apply room specific maintenance charges.*

- **Record flat details;** room number and associated owner and the space type (apartment size).
- **Autofills charges;** from the charges table once the spacetype is filled in.
- **In sync with the charges table;** any new changes in the charges table will reflect in the existing residents details.
- **Toggle lease/rent charges on or off.**
- **Autofills parking charge;** once the number of vehicles is selected.
- **Columns to add pending penalties or uncleared balances.**
- **Live total;** keeps working as the details are being filled in.
- **Add/remove rows** to record new rooms or **delete existing rows.**
- **Bulk deletion,** delete multiple rows with a single click.
- **Built in data validation;** ensures required fields are filled.
- **Auto save/update;** all changes save immediately, no saves required.
- **Auto refresh;** no manual refresh needed, but still present.

Log Payments: *Record maintenance payment’s details for existing residents.*

- **Select a room number from the list of existing residents.**
- **Autofills information,** which is recorded in the Resident’s Maintenance charges table; upon room number entry.
- **Enter the number of months the payment covers** and the applied charges adjust accordingly.
- **Enter payment details** such as payment date, payment mode, amount etc.
- **Add Penalty for late payments;** any pending penalties or uncleared balance will also be applied but can be removed.
- **The date of the entry is also recorded and mentioned in the invoice.**
- **Add/remove rows** to record new payments or **delete existing unsaved rows.**
- **Bulk deletion;** delete multiple unsaved rows with a single click.
- **Built in data validation;** ensures required fields are filled.
- **Saves are user initiated;** save only when all details for a single or multiple payments are completed.
- **Activity log** to notify users of any success/failure during the room entry, batch creation and saving processes.

View Payments Record: *View and edit all the payments logged through the payload table.*

- **View any payment** from the list of all the recorded payments.
- **Filter for the payments list** (available filters: room no., owner, payment amount, payment-date, record-date)
- **Edit any payment details** in the existing record.
- **Saves are user initiated;** save changes made to any payment details.
- **Delete** any payments from the record.
- **View receipt;** for recorded payments.
- **Bulk deletion;** delete multiple unsaved rows with a single click.
- **Bulk View;** View multiple payment receipts.
- **Bulk print;** Print all selected payment receipts.
- **Receipts can be configured to add building registration details and any notes, also add any signature image.**

Functional flow:

1. Define Spacetypes → specify amount to be paid per charge per spacetype →add any additional charges if needed.
2. Enter flat and resident details → Amount for defined charges get filled in →
3. Enter and save payment details.
4. View records: if needed make any edits → Save any edits made →View receipts.
5. Configure receipt → add registration details of the society, notes and signature of treasurer or chairperson.

1. Define Spacetypes →

Spacewise ChargesMaintenance DataRecord PaymentsView Records

Define standard maintenance charges for each space type.

+ Add New Space Type

Spacewise charges loaded.

Space Type

Building Maintenance

Repair Funds

Sinking Funds

Parking (JSON)

Lease/Rent

Enter Charge

set

undefined

0.00

0.00

0.00

{ }

0.00

0.00

specify amount to be paid per charge per spacetype → add any additional charges if needed.

+ Add New Space Type

Data saved successfully!

Space Type

Building Maintenance

Repair Funds

Sinking Funds

Parking (JSON)

Lease/Rent

Enter Charge

set

1 BHK

750

150

50

"1": 30,
"2":90
}

250

0.00

Security Charges

set

150

+ Add New Space Type

Spacewise charges loaded.

Space Type

Building Maintenance

Repair Funds

Sinking Funds

Parking (JSON)

Lease/Rent

Security Charges

remove

Enter Charge

set

1 BHK

750.00

150.00

50.00

{
"0": 0,
"1": 30,

250.00

150.00

0.00

2. Select a spacetype from the list →

Spacewise ChargesMaintenance DataRecord PaymentsView Records

Manage maintenance details for individual rooms. Charges will auto-fill based on Space Type.

+

Data loaded successfully!

☐	Room No.	Owners	Space Type	Building Maintenance	Repair Funds	Sinking Funds	Security Charges	Num Vehicles	Parking
☐			Select Space Type First 1 BHK	0.00	0.00	0.00	0	0 Vehicles	0.00

Amount for defined charges get filled in →

Data loaded successfully!

Space Type	Building Maintenance	Repair Funds	Sinking Funds	Security Charges	Num Vehicles	Parking	Lease/Rent
1 BHK	750.00	150.00	50.00	150.00	0 Vehicles	0.00	250.00

Enter room and resident details →

☐	Room No.	Owners	Space Type
☐	101	Mr. XYZ	1 BHK

Select a Number of vehicles form the list (list consists all the defined keys from charges table) →

Num Vehicles	Parking
0 Vehicles	0.00
0 Vehicles	
1 Vehicle(s)	
2 Vehicle(s)	

Select whether to apply charges for lease/rent →

Lease/Rent	Apply Lease/Rent	Penalty	Balance	Total
250.00	Yes	0.00	0.00	1350.00
	Yes			
	No			

3. Add a new row →

Spacewise ChargesMaintenance DataRecord PaymentsView Records

Log maintenance payments. Enter Room No. to auto-fill details from Maintenance Data.

+

Ready to log payments.

Activity Log

Ready for logging.

Enter and save payment details. →

+

Data auto-filled for Room: 101

☐	Room No	Owners	Date Made	Months	Period Covered	Space	PBM	RMF	Sink
☐	101	Mr. XYZ	11-10-2025	1	e.g., AUG-SEP 2024	1 BHK	750.00	150.00	

Activity Log

Ready for logging.Attempting to submit 1 payments.
Batch payment successful. Total records created: 1

4. View records: if needed make any edits → Save any edits made →View receipts.

Spacewise ChargesMaintenance DataRecord PaymentsView Records

☐

Payment Records

▼

Filter Options

▼

☐

101 : Mr. XYZ
Oct 11, 2025 | OCT'25

₹110.00
CASH

Payment Details

Payment deleted successfully

Charges Breakdown

Periodic Building Maintenance:₹750.00

Repair & Maintenance Fund:₹150.00

Sinking Funds:₹50.00

Security Charges:₹150.00

Parking:₹0.00

Penalty:₹0.00

Lease Rent:₹0.00

Balance:₹0.00

Total Maintenance Amount:₹1100.00

Payment Information

Payment Amount:₹110.00

Payment Date:Oct 11, 2025

Payment Mode:CASH

Receipt Date:Oct 11, 2025

Record Information

Record ID:#34

EditDeleteView Receipt

5. Configure receipt → add registration details of the society, notes and signature of treasurer or chairperson.

SOCIETY

Address:
Registration No:
PAN Number:

Name:	Mr. XYZ	Invoice No:	34
Flat No.:	101	Date:	11-10-2025
		Months:	1

Maintenance Charges

PERIOD: OCT'25

Sr. No.	Description	Amount
1	Periodic Building Maintenance	750.00
2	Repair & Maintenance Fund	150.00
3	Sinking Funds	50.00
4	Security Charges	150.00
5	Parking	0.00
6	Penalty/Interest	0.00
7	Lease Rent	0.00
8	Previous Balance	0.00
GRAND TOTAL		1100.00

Amount in Words:

One Hundred Ten Rupees Only

Sr. No.	Payment Details	
1	Payment Mode: CASH	Configure Settings (Secretary / Chairman /
2	Payment Date: 11-10-2025	
3		

Print Receipt

Design:

- **Backend:** Runs on Django, uses DRF for setting up api routes.
- **Frontend:** Uses Vanilla JS...backend renders a home.html template file which loads static assets (Page JS and CSS files) linked to it.
- **Model:**
 - 1. SpacewiseCharges Model:**

This model defines the base charges for a given type of space (for example: 1BHK, 2BHK, Shop, or any other user defined input.). It stores maintenance charges like:
Building maintenance, Repair fund, Sinking fund, Lease/rent, Parking (as a flexible JSON, since different vehicles/spaces may have different rates)
Any other additional charges (also in JSON for flexibility)(Yet to be Implemented in deployed ver.)
Other models can reference this to calculate actual maintenance for individual rooms.
 - 2. MaintenanceData Model:**

It is directly linked to SpacewiseCharges (so rates can be auto-filled based on space type).
However, it still stores actual values separately (to lock in charges at entry time and allow manual overrides).
Adds more flexibility:
Whether lease rent applies or not (boolean switch)
Number of vehicles (to calculate parking)
Penalty, balance, and total for each room,
it combines base rate references and room-specific adjustments.
 - 3. PayLog Model:**

Tracks payments made by owners.
For each payment record, it keeps:
Room number and owner details
Date of receipt and payment
What months/period are covered by the payment
Breakdown of charges (maintenance, repair fund, sinking, parking, penalty, lease rent, etc.)
Payment details (mode, cheque info if applicable, notes)
How much was paid vs. total owed (balance/advance)
Serves as the transaction history / ledger for maintenance collections.
These Payments will then be displayed in the form of a premade template HTML file.
- **API endpoints:**

For CRUD ops.
Django REST Framework (DRF) to expose backend models as API endpoints. These endpoints allow the frontend to fetch, create, update, and delete data via AJAX calls.
The APIs are structured using DRF ViewSets and a DefaultRouter, which automatically generates URL patterns for common CRUD operations.
Uses Rest Convention, allowed Requests: GET, POST, PUT, DELETE

The following resources are exposed:

Maintenance Data: /api/maintenance-data-new/
Backed by MaintenanceDataNewViewSet
Provides CRUD operations on the newer, structured per-room records that reference base charges.

Spacewise Charges: /api/charges/
Backed by SpacewiseChargesViewSet
Exposes the rate card data (base charges for each type of space).

Pay Logs: /api/pay-log/
Backed by PayLogReViewSet
Provides transaction/payment history records.
- **Handling SPA:**

Pages were first made one at a time with separate nav-link for each page, to avoid reloading the entire page when switching between sections, HTML components of each page were added into separate <div>'s and rolled into one single HTML file.

Router Function:
It works by changing the display property of <style> tag, thus controlling which page section is visible to the user depending on the URL hash.

How it Works:

 - 1. Hide all pages first**

All elements whose id starts with "page-" are hidden (display: none).
This makes sure only one section is ever visible at a time.
 - 2. Read the current hash in the URL**

Example:
/#charges → shows the charges page
/#maintenance → shows the maintenance page
If no hash is given, it defaults to the charges page.
 - 3. Show the correct page**

Finds the element that matches the current hash (like id="page-charges")
Sets its style to display: block so it becomes visible.

4. Page Initializers

After making the section visible, the router calls a page-specific initialization function, depending on which page is active:
initChargesPage() → loads charges data & applies charges.css
initMaintenancePage() → loads maintenance data & applies index.css
initPaymentsPage() → sets up adding new payment entries & applies paylog.css
initPaymentRecordsPage() → loads payment history records & applies paylog_view.css

Each initializer:
Switches to the correct CSS file for that page (switchStyle(...)). &
Calls the proper fetch function to load data from your DRF backend API (fetchSpacewiseCharges, fetchInitialMaintenanceData, fetchPayments...).

5. Listeners

The router is triggered:
On page load (window.addEventListener("load", router))
Whenever the URL hash changes (window.addEventListener("hashchange", router))
This makes navigation feel instant, without reloading the page.

• Pages:

The Pages for the WebApp are also designed in correspondence with the model:
Brief Description:
Charges Page: This is where user can define a standard space type for the apartment (eg: 2BHK) and define charges associated with that specific space type. (Periodic maintenance charges for 2BHK is 780.00).
Maintenance page: This is where the user enters resident's information: their room number, space type, associated charges will be autofilled upon selection of space type (using the data from the charges db), then any additional charges eg: parking, lease or rent, penalty, advance or balance in payments.
Payments page: Used to log any payments made, just enter the room number and relevant information is auto-filled...user then fills in payment data (eg: payment mode, payment amount, payment date).
Payment records: User can view and edit payment logs here... and choose to delete them and a receipt can also be viewed and printed for record keeping. Receipt information can be changed through a configure option... this allows user to add any information of the apartment complex and add signature for the person incharge of managing maintenance.

• Page Functions:

Each page has some common functions which are to be implemented to make these async CRUD operations as seamless as possible:

Fetching:

All pages use the same method for fetching i.e:
Fetched data from the API comes as a JSON array of objects.
Example (GET /api/charges/):

```
json
[
  {
    "id": 1,
    "space_type": "1 BHK",
    "periodic_building_maintenance": "750.00",
    "repair_and_maintenance_fund": "150.00",
    "sinking_funds": "70.00",
    "lease_rent": "250.00",
    "parking": {
      "0": 0,
      "1": 30,
      "2": 90,
      "3": 150
    }
  },
  {
    "id": 2,
    "space_type": "2 BHK",
    "periodic_building_maintenance": "780.00",
    "repair_and_maintenance_fund": "150.00",
    "sinking_funds": "100.00",
    "lease_rent": "250.00",
    "parking": {
      "0": 0,
      "1": 30,
      "2": 90,
      "3": 150
    }
  }
]
```


Displaying Data:

This array is looped, and each object becomes a <tr> row in the rendered table.
If array length = 0 → insert one empty object {} to seed the table for new data entry.

Adding Table:

Row data inside a table is always represented as a JavaScript Object, where:
Keys = field names (space_type, room, penalty, balance, etc.)
Values = typed data (string, float, int, JSON object, boolean)

Eg: in maintenance data page
`const fields = [
 { name: 'room', type: 'text', value: rowData.room || '', disabled: !isExistingDataWithSpaceType },
 { name: 'owners', type: 'text', value: rowData.owners || '', disabled: !isExistingDataWithSpaceType },

 { name: 'balance', type: 'number', value: parseFloat(rowData.balance || 0).toFixed(2), disabled: !isExistingDataWithSpaceType },
 { name: 'total', type: 'number', value: parseFloat(rowData.total || 0).toFixed(2), readonly: true }
];`

Field are then added by looping and creating a cell : <td> DOM element based on the type of field, assigning name and value properties the same way.
rowData is the object in which fetched JSON is stored and can be its propertioes can be accessed by using the dot notation(.propertyName)
eg:{ name: 'room', type: 'text', value: rowData.room || '', disabled: !isExistingDataWithSpaceType },

Validation:

Required fields validated client-side before saving.
Prevents editing other fields before UI disables fields are selected.
New inputs trigger recalculation of total live.

Saving:

When a user edits a field inside a row, all editable inputs in the same row are collected into a single object (representing ONE model row).
This object is serialized with JSON.stringify() and sent via fetch() to DRF.
DRF either creates (POST) or updates (PUT) a single record in the backend DB.
Backend returns the saved record (as JSON).
API Call:
POST /api/maintenance-data-new/ if new.
PUT /api/maintenance-data-new/{id}/ if editing.
On success:
DOM row dataset updated with API-assigned ID.
Row values refreshed from backend response (ensures backend-calculated totals/parking are synced).

Deleting:

Delete button reveals a Bootstrap modal confirmation (for persisted rows).
On confirm → DELETE /api/maintenance-data-new/{id}/.
On success → DOM <tr> is removed.
If no ID (unsaved row) → simply removes DOM node.

How the reciept template works:

Upon selecting a payment record user can see a view receipt button which onclick executes the view_receipt() function.
A view receipt function exists in the payments record page.
This stores the fetched data from the paylog in the session storage as a JSON object and triggers *window.open(reciept.html)*
The receipt template is just a predefined HTML tables for charges, totals, notes, etc. and the receipt-specific data is pulled from sessionStorage, while society configuration is pulled from localStorage.
The receipt can also be configured as per user need:
Configurable fields are: society name, address, registration num, PAN, notes, signature(img).

Also, added:

Custom charges:

This will allow user to have custom charges which would be stored as key/value pair in the additional_charges field eg; {“cleaning charges”:50}
This will allow the user to enter a charge name in an input field situated inside of a table head(this would be the key) and table cells (value for each row).

Bulk Printing:

Selecting multiple payment records and print all receipts.